

ServiceNow JavaScript Reference Manual

Detailed Scratch-to-Advanced Concepts & Code Examples

This reference manual covers core JavaScript concepts and ServiceNow client-server APIs. Every language element includes detailed explanations, practical code fragments, and environment details. Keep this document handy when designing and writing ServiceNow applications.

1. The ServiceNow JavaScript Environments

ServiceNow splits JavaScript execution between client-side scripts (runs in browser) and server-side scripts (runs directly on the cloud server database).

Feature	Client-Side JS	Server-Side JS
Where it runs	User's Web Browser (Chrome, Safari, Edge, etc.)	ServiceNow Cloud Server (Mozilla Rhino JS engine).
Main Purpose	UI interactions, form validations, showing/hiding fields dynamically.	Database CRUD queries, business rules, API scripts, workflows.
Key APIs	g_form, g_user, GlideAjax	GlideRecord, gs (GlideSystem), GlideDateTime, GlideAggregate
Performance	High impact on browser UI responsiveness (avoid freezing user).	High impact on database execution speeds and server memory.

2. JavaScript Fundamentals

A. Variables & Declarations

1. var (Function-Scoped)

var is declared within a function and accessible anywhere in that function. It does not respect block scoping (e.g. within an 'if' block or 'for' loop). Hoisting allows var to be referenced before declaration, which can lead to bugs.

```
function testVar() {  
  if (true) {  
    var x = 10; // var ignores the 'if' block scope  
  }  
  gs.info(x); // Works! Prints 10 because var is function-scoped  
}
```

2. let (Block-Scoped, Reassignable)

let is block-scoped. It is only accessible inside the curly braces '{}' it was defined in. This prevents variable leakage. It can be updated or reassigned.

```
function testLet() {  
  if (true) {  
    let y = 20; // y is restricted to this 'if' block  
  }  
  // gs.info(y); // ERROR! y is undefined outside the block  
}
```

3. const (Block-Scoped, Read-Only)

const stands for constant. Like let, it is block-scoped. However, once assigned a value, it cannot be reassigned or redefined. Use const for configuration constants or database table names.

```
const tableName = 'incident';  
// tableName = 'change_request'; // ERROR! Cannot reassign a constant
```

B. Data Types

JavaScript is dynamically typed. Variables hold values of different data types:

1. String: Text character lists, wrapped in quotes ('Incident' or "Active").
2. Number: Numeric values, including decimals and integers (100, 3.14).
3. Boolean: Logical flags that can only evaluate to true or false.
4. Null: Represents the intentional absence of an object value.
5. Undefined: A variable that has been declared but not assigned any value.

```
var incidentNum = 'INC0001';    // String
var priorityValue = 1;          // Number
var isClosed = false;           // Boolean
var clearedField = null;        // Null
var emptyVar;                   // Undefined (value defaults to undefined)
```

6. Object: Containers holding multiple properties represented as key-value pairs.
7. Array: Ordered lists of index-based items (0-indexed).

```
// Object Example
var incidentInfo = {
  number: 'INC0001',
  priority: 1,
  active: true
};
gs.info(incidentInfo.number); // Access property: 'INC0001'

// Array Example
var categories = ['Hardware', 'Software', 'Network'];
gs.info(categories[0]); // Access index 0: 'Hardware'
```

C. Operators

1. Arithmetic Operators:

Used to perform calculations: + (add/concatenate), - (subtract), * (multiply), / (divide), % (modulus: returns division remainders).

```
var sum = 10 + 5;           // sum is 15
var remainder = 10 % 3;     // remainder is 1
var msg = 'INC: ' + 1024;   // Text concatenation: 'INC: 1024'
```

2. Assignment Operators:

Used to assign values. Combined shorthand operators perform arithmetic and assignment in one step.

```
var count = 5;
count += 2; // Equivalent to: count = count + 2; (count becomes 7)
```

3. Comparison Operators:

Compare two operands and return boolean results. Note the difference between loose equality == (values only, performs type conversion) and strict equality === (values and data types must match).

```
5 == '5'; // true (interprets string '5' as number 5)
5 === '5'; // false (number is not strictly equal to string type)
5 !== 10; // true (checks inequality)
```

4. Logical Operators:

Used to join comparisons: && (Logical AND: both true), || (Logical OR: at least one true), ! (Logical NOT: reverses boolean).

```
var isP1 = true;
var isActive = false;
var check1 = isP1 && isActive; // false (both must be true)
var check2 = isP1 || isActive; // true (at least one is true)
var opposite = !isP1;         // false
```

D. String Methods

Common string methods used to check fields or parse email/text inputs in ServiceNow:

- `indexOf(val)`: Finds starting index of search string. Returns -1 if not found.
- `includes(val)`: Returns true if text exists in string, false otherwise.
- `toLowerCase()` / `toUpperCase()`: Conversions for standard comparison check.
- `split(char)`: Divides string at character separator to return an array of items.
- `replace(old, new)`: Finds the first occurrence of text and replaces it.
- `trim()`: Strips off trailing or leading whitespaces (spaces, tabs, newlines).
- `substring(start, end)`: Extracts characters starting at index 'start' up to (exclusive) 'end'.

```
var description = '  Incident INC0001024 has failed.  ';\n\n// 1. Clean whitespace and convert to upper case\nvar cleanDesc = description.trim().toUpperCase();\n// cleanDesc is now: 'INCIDENT INC0001024 HAS FAILED.'\n\n// 2. Locate ID using substring and indexOf\nif (cleanDesc.includes('FAILED')) {\n  var index = cleanDesc.indexOf('INC');\n  var incId = cleanDesc.substring(index, index + 10); // Extracts 'INC0001024'\n}
```

E. Array Methods

Arrays organize collections like Sys IDs, states, or selected choices. Common operations:

- `push(item)`: Adds elements to the end of the array.
- `pop()`: Removes and returns the last element.
- `shift()`: Removes and returns the first element, shifting older index elements down.
- `unshift(item)`: Prepends elements to the beginning of the array.
- `indexOf(item)`: Returns the index of the first occurrence of an item (returns -1 if missing).
- `join(char)`: Merges all array items into a single string separated by character.
- `forEach(callback)`: Executes callback block for each array item.
- `filter(callback)`: Returns a new array with items that pass criteria checks in callback.

```
var states = ['New', 'In Progress'];
states.push('On Hold'); // states becomes: ['New', 'In Progress', 'On Hold']

var lastState = states.pop(); // Returns 'On Hold', array is: ['New', 'In Progress']

var firstState = states.shift(); // Returns 'New', array is: ['In Progress']

var roles = ['admin', 'itil', 'user'];
var itilIndex = roles.indexOf('itil'); // Returns 1

// Filter numbers less than or equal to 2
var priorities = [1, 2, 3, 4, 5];
var criticals = priorities.filter(function(p) {
    return p <= 2;
}); // criticals becomes: [1, 2]

var textCsv = roles.join(','); // 'admin,itil,user'
```

F. Conditions (Control Flow)

Conditions control code execution paths based on Boolean flags:

- if/else if/else: Standard conditional logic branches.
- switch: Compares values against cases. Best replaced for multiple choice parameters.
- Ternary: Shorthand syntax for simple assignments: condition ? value_if_true : value_if_false.

```
var priority = 1;
if (priority === 1) {
    gs.info('P1 Critical!');
} else if (priority === 2) {
    gs.info('P2 High');
} else {
    gs.info('Standard Priority');
}

// Switch Case
var category = 'software';
switch (category) {
    case 'hardware': gs.info('Hardware team'); break;
    case 'software': gs.info('Software team'); break;
    default: gs.info('Service Desk');
}

// Ternary Shorthand
var active = (priority === 1) ? true : false;
```

G. Loops

Loops repeat blocks of code until a condition is met:

- while: Loops while condition evaluates to true. (Standard GlideRecord query loops).
- for: Classic loop using initialization, check condition, and counter update.
- for...in: Iterates over the properties (keys) of an object.
- for...of: Iterates directly over the values of arrays or list items.

```
// While Loop (GlideRecord database looping structure)
var gr = new GlideRecord('incident');
gr.query();
while (gr.next()) { ... }

// Classic For Loop
for (var i = 0; i < 3; i++) {
    gs.info('Index: ' + i);
}

// For...in Loop (Objects)
var user = { name: 'Rupam', role: 'admin' };
for (var key in user) {
    gs.info(key + ' = ' + user[key]);
}

// For...of Loop (Arrays)
var colors = ['red', 'blue'];
for (var color of colors) {
    gs.info(color);
}
```

H. Functions

Functions are callable blocks of code. They support declarations, expressions, or callback arguments:

```
// 1. Function Declaration (Hoisted to top of scope)
function getGreeting(name) {
    return 'Welcome, ' + name;
}

// 2. Function Expression (Not hoisted)
var calculateTax = function(subtotal, rate) {
    return subtotal * rate;
};

// 3. Callback Function (Passed as parameters, common in GlideAjax)
function executeTask(callback) {
    gs.info('Running logic...');
    callback('Task Complete!');
}
executeTask(function(message) {
    gs.info('Callback returned: ' + message);
});
```

3. Server-Side ServiceNow JavaScript (The Glide APIs)

A. GlideRecord Queries (Read)

```
var incidentGR = new GlideRecord('incident'); // 1. Target table
incidentGR.addQuery('active', true);           // 2. Add query parameter filters
incidentGR.addQuery('priority', 1);
incidentGR.query();                           // 3. Send query request to SQL engine

// 4. Loop records returned using next()
while (incidentGR.next()) {
    gs.info('Found Active P1: ' + incidentGR.getValue('number'));
}
```


B. GlideRecord Database Operations (Create, Update, Delete)

```
// 1. CREATE RECORD
var newRecord = new GlideRecord('incident');
newRecord.initialize(); // Set default values
newRecord.setValue('short_description', 'Database connection issue');
newRecord.setValue('urgency', 1);
var sysId = newRecord.insert(); // Insert to DB and return primary sys_id

// 2. UPDATE RECORD
var updateGR = new GlideRecord('incident');
if (updateGR.get(sysId)) { // Query a single record directly by sys_id
    updateGR.setValue('state', 2); // In Progress
    updateGR.update();           // Save changes
}
```

C. GlideAggregate (Efficient Summaries)

GlideAggregate executes calculations directly inside the SQL database engine instead of loading thousands of records into server memory:

```
var agg = new GlideAggregate('incident');
agg.addQuery('active', true);
agg.addAggregate('COUNT');
agg.query();

var count = 0;
if (agg.next()) {
    count = agg.getAggregate('COUNT');
}
gs.info('Active Incidents Count: ' + count);
```

NOTE: Never use `GlideRecord.getRowCount()` on tables with large amounts of data just to get counts. It is memory heavy. Always use `GlideAggregate COUNT`.

D. GlideSystem (gs) Utility APIs

Provides access to server metrics, logs, properties, and logged-in user context parameters:

```
// 1. Logger statements (Review logs at System Logs -> syslog table)
gs.info('Standard log message');
gs.warn('System warning message');
gs.error('Script failure message');

// 2. Logged-in user context details
var userSysId = gs.getUserID();           // Returns sys_id of current user
var isITIL = gs.hasRole('itil');          // Checks user permissions

// 3. Properties settings
var maxLimit = gs.getProperty('my.custom.max_limit', '50');
```

E. GlideDateTime (Date/Time mathematics)

ServiceNow records dates and times in standard UTC (GMT). Use GlideDateTime for adding/subtracting dates and printing values in timezone formats:

```
var gdt = new GlideDateTime(); // Current UTC date and time
gdt.addDaysLocalTime(5);       // Add 5 days based on local timezone settings
gs.info(gdt.getDisplayValue()); // Prints local timezone display format
```

4. Client-Side ServiceNow JavaScript (The Form APIs)

Client-side scripts run inside user browsers. Use them to make form interactions dynamic.

A. g_form (GlideForm) Layout Functions

```
var shortDesc = g_form.getValue('short_description');
g_form.setValue('short_description', 'Auto: ' + shortDesc);

// Manage field layouts
g_form.setMandatory('work_notes', true); // Field is required
g_form.setReadOnly('assigned_to', true); // Lock field selection
g_form.setDisplay('comments', false);    // Hide comments field
```

B. g_user (GlideUser) Parameters

```
var firstName = g_user.firstName;
if (g_user.hasRole('itil')) {
    g_form.addInfoMessage('Logged in as agent: ' + firstName);
}
```

5. Reusable Code: Script Includes (OOP)

Script Includes are reusable server-side classes. Use them to encapsulate logic.

```
var IncidentUtils = Class.create();
IncidentUtils.prototype = {
    initialize: function() {
        // Class constructor
    },
    checkAssignment: function(incidentGR) {
        var assigned = incidentGR.getValue('assigned_to');
        if (!assigned) return false;
        var user = new GlideRecord('sys_user');
        if (user.get(assigned)) {
            return user.getValue('active') == '1';
        }
        return false;
    },
    type: 'IncidentUtils'
};
```

6. Scripting Performance & Best Practices

1. Avoid GlideRecord on the Client Side:

Using new GlideRecord() in client scripts runs synchronous database queries. This freezes the user's browser window until ServiceNow responds. Always use GlideAjax + Script Includes for client-to-server operations.

2. Avoid Database Queries in Loops (N+1 Query Issue):

Do not run a GlideRecord query inside a while loop. It generates repetitive, round-trip SQL executions that slow down the server. Collect matching record IDs into an array, and use a single query with addQuery('sys_id', 'IN', array_of_ids) instead.

3. Safe Query Limits:

Always use setLimit(N) to restrict returned records when querying massive transaction logs (syslog) or audit tables.

4. Safe Logging limits:

Delete temporary gs.info() and client-side jslog() statements before pushing code to Production. Excess logging occupies database storage and degrades system performance.

NOTE: Verify that all database queries check indexed fields, ensure no queries are called within loops, and enclose complex code in standard try/catch error handling blocks.